

1. Write a python programme to solve 8 puzzle problem

Aim:

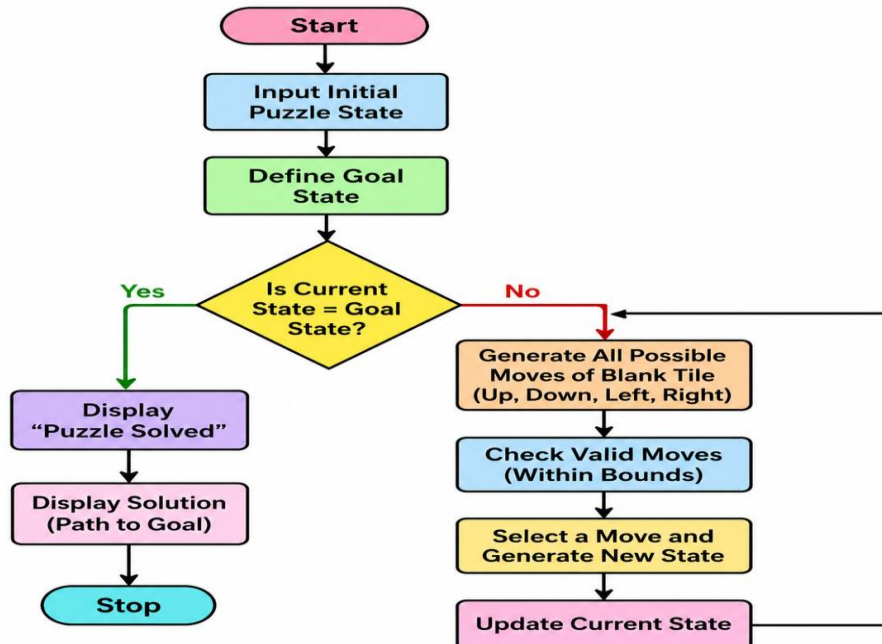
Write a program to solve 8 puzzle problem.

Objectives: The 8-puzzle problem is a game with 8 numbered tiles and 1 blank space (0) arranged in a 3×3 grid. The objective is to move the blank space up, down, left, or right to arrange the tiles in the correct order and reach the goal state.

Algorithm:

1. Start.
2. Define the initial puzzle state.
3. Define the goal state.
4. Compare the current state with the goal state.
5. If they are the same, display "Puzzle Solved".
6. Otherwise, move the blank tile (0) in a valid direction.
7. Repeat steps 4–6 until the goal state is reached.
8. Stop.

Flowchart:



Source Code:

```
from collections import deque

goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

start = ((1, 2, 3),
        (4, 0, 6),
        (7, 5, 8))

def get_next_states(state):
    states = []
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [list(row) for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            states.append(tuple(tuple(row) for row in new))

    return states

def solve(start):
    queue = deque([start])
    visited = set()

    while queue:
        state = queue.popleft()

        if state == goal:
            print("Puzzle Solved!")
            for row in state:
                print(row)
            return

        visited.add(state)
```

```

for next_state in get_next_states(state):
    if next_state not in visited:
        queue.append(next_state)

```

```

print("No Solution Found")

```

solve(start)

Result:

The program was executed successfully. The blank tile (0) was moved correctly, and the initial puzzle state was transformed into the goal state. Hence, the 8-puzzle problem was solved successfully.

Output:

```

Programiz
Python Online
Compiler

main.py  Run  Output  Clear

1 from collections import deque
2 goal = ((1, 2, 3),
3         (4, 5, 6),
4         (7, 8, 0))
5
6 start = ((1, 2, 3),
7         (4, 0, 6),
8         (7, 5, 8))
9
10
11 def get_next_states(state):
12     states = []
13
14     for i in range(3):
15         for j in range(3):
16             if state[i][j] == 0:
17                 x, y = i, j
18
19                 moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
20
21                 for dx, dy in moves:
22                     nx, ny = x + dx, y + dy
23                     if 0 <= nx < 3 and 0 <= ny < 3:
24                         new_state = [list(row) for row in
25                                     state]
26                         new_state[x][y], new_state[nx][ny] =
27                             new_state[nx][ny],
28                             new_state[x][y]
29                         states.append(tuple(tuple(row) for
30                                             row in new_state))
31
32     return states
33
34 def bfs(start):
35     queue = deque([(start, [])])
36     visited = set()
37
38     while queue:
39         state, path = queue.popleft()
40
41         if state == goal:
42             return path + [state]
43
44         if state in visited:
45             continue
46         visited.add(state)
47
48         for next_state in get_next_states(state):
49             queue.append((next_state, path +
50                           [state]))
51
52     return None
53
54 solution = bfs(start)
55
56 if solution:
57     print("Solution found:")
58     for step in solution:
59         for row in step:
60             print(row)
61         print()
62 else:
63     print("No solution found.")

```

Output

```

Solution found:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

```

Code Execution Successful

2. Write a python program to solve 8 Queen's problem

Aim: To write a Python program to solve the 8-Queen Problem

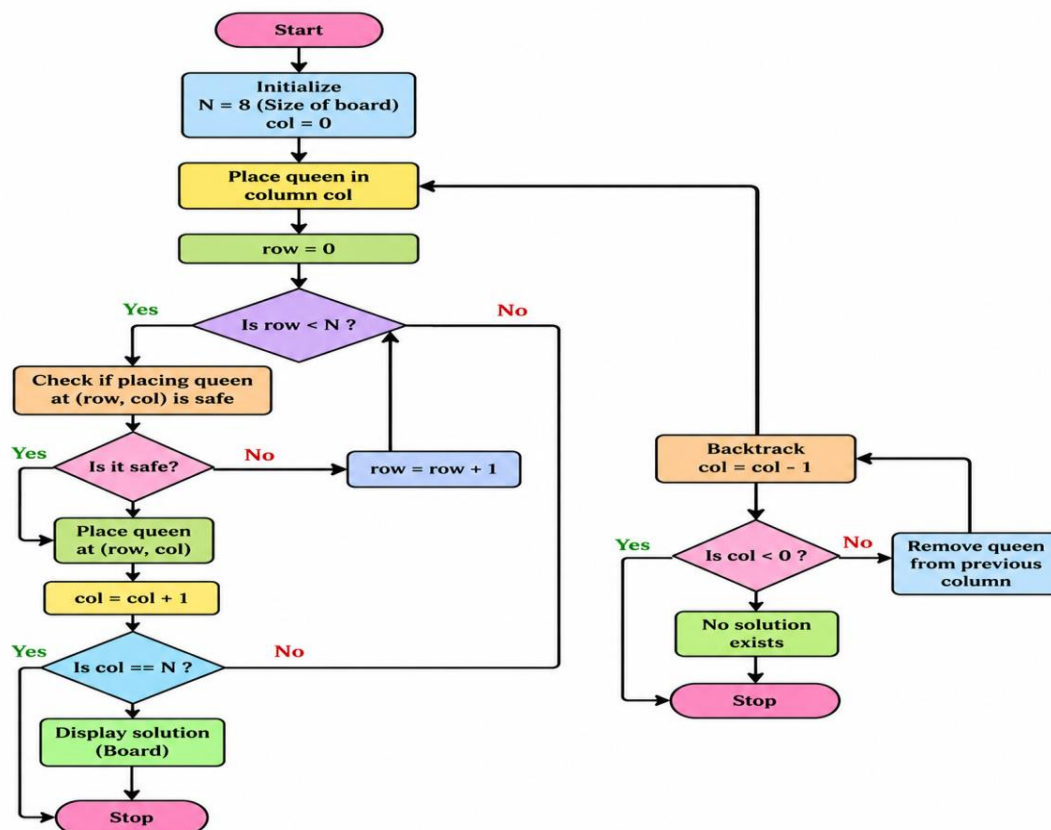
Objectives:

The objective of this program is to develop a Python program to solve the 8-Queen Problem by placing eight queens on an 8×8 chessboard so that no two queens attack each other. The program checks the rows, columns, and diagonals to find a valid arrangement and displays the solution.

Algorithm:

1. Start.
2. Create an empty 8 × 8 chessboard.
3. Place a queen in the first column.
4. Check whether the position is safe (row and diagonals).
5. If safe, place the queen and move to the next column.
6. If no safe position is found, backtrack and try another row.
7. Repeat until all 8 queens are placed.
8. Display the chessboard with the queen positions.
9. Stop.

Flow chart:



Source Code:

N = 8

```
board = [[0] * N for _ in range(N)]
```

```
def is_safe(row, col):  
    for i in range(col):  
        if board[row][i] == 1:  
            return False
```

```
    i, j = row, col  
    while i >= 0 and j >= 0:  
        if board[i][j] == 1:  
            return False  
        i -= 1  
        j -= 1
```

```
    i, j = row, col  
    while i < N and j >= 0:  
        if board[i][j] == 1:  
            return False  
        i += 1  
        j -= 1
```

```
    return True
```

```
def solve(col):  
    if col >= N:  
        return True
```

```
    for i in range(N):  
        if is_safe(i, col):  
            board[i][col] = 1  
            if solve(col + 1):  
                return True  
            board[i][col] = 0
```

```
    return False
```

```
if solve(0):  
    for row in board:  
        print(row)  
else:  
    print("No Solution")
```

The program was executed successfully, and a valid arrangement of 8 queens was obtained on the 8×8 chessboard such that no two queens attack each other.

main.py

Run

```
1 N = 8
2
3 board = [[0] * N for _ in range(N)]
4
5 def is_safe(row, col):
6     for i in range(col):
7         if board[row][i] == 1:
8             return False
9
10    i, j = row, col
11    while i >= 0 and j >= 0:
12        if board[i][j] == 1:
13            return False
14        i -= 1
15        j -= 1
16
17    i, j = row, col
18    while i < N and j >= 0:
19        if board[i][j] == 1:
20            return False
21        i += 1
22        j -= 1
23
24    return True
25
26 def solve(col):
27     if col >= N:
28         return True
29
30     for i in range(N):
31         if is_safe(i, col):
32             board[i][col] = 1
33             if solve(col + 1):
34                 return True
35             board[i][col] = 0
36
37     return False
38
39 if solve(0):
40     for row in board:
41         print(row)
42 else:
43     print("No Solution")
```

Output

[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

=== Code Execution Successful ===

3. Write a python program to solve water jug problem

Aim:

To write a Python program to solve the Water Jug Problem.

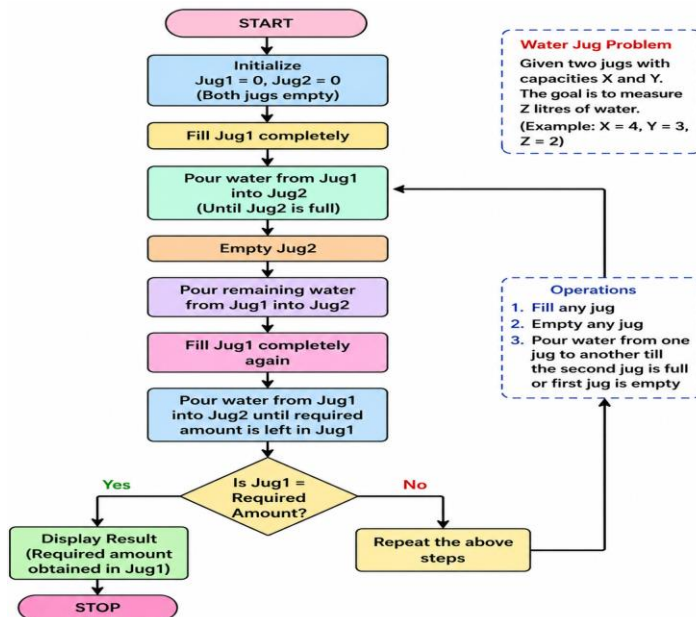
Objectives:

The objective of this program is to solve the Water Jug Problem by measuring the required amount of water using two jugs of different capacities through filling, emptying, and pouring operations.

Algorithm:

1. Start.
2. Initialize both jugs as empty.
3. Fill the first jug completely.
4. Pour water from the first jug into the second jug until the second jug is full.
5. Empty the second jug.
6. Pour the remaining water from the first jug into the second jug.
7. Fill the first jug again.
8. Pour water into the second jug until the required amount is left in the first jug.
9. Display the result.
10. Stop.

Flow chart:



Source Code:

```
# Water Jug Problem
```

```
jug1 = 0
```

```
jug2 = 0
```

```
while jug1 != 2:
```

```
    if jug1 == 0:
```

```
        jug1 = 4
```

```
        print(jug1, jug2)
```

```
    elif jug2 == 3:
```

```
        jug2 = 0
```

```
        print(jug1, jug2)
```

```
    else:
```

```
        transfer = min(jug1, 3 - jug2)
```

```
        jug1 -= transfer
```

```
        jug2 += transfer
```

```
        print(jug1, jug2)
```

```
print("Goal Achieved: 2 Litres in Jug1")
```

Result:

The program was executed successfully, and the required quantity of water was measured using the two jugs. The goal state was achieved successfully.

Output:

main.py

Run

Output

Clear

```
1 # Water Jug Problem
2
3 jug1 = 0
4 jug2 = 0
5
6 while jug1 != 2:
7     if jug1 == 0:
8         jug1 = 4
9         print(jug1, jug2)
10
11     elif jug2 == 3:
12         jug2 = 0
13         print(jug1, jug2)
14
15     else:
16         transfer = min(jug1, 3 - jug2)
17         jug1 -= transfer
18         jug2 += transfer
19         print(jug1, jug2)
20
21 print("Goal Achieved: 2 Litres in Jug1")
```

```
4 0
1 3
1 0
0 1
4 1
2 3
Goal Achieved: 2 Litres in Jug1
=== Code Execution Successful ===
```

4. Write a python program to solve vacuum cleaner problem

Aim:

To write a python program to solve vacuum cleaner problem .

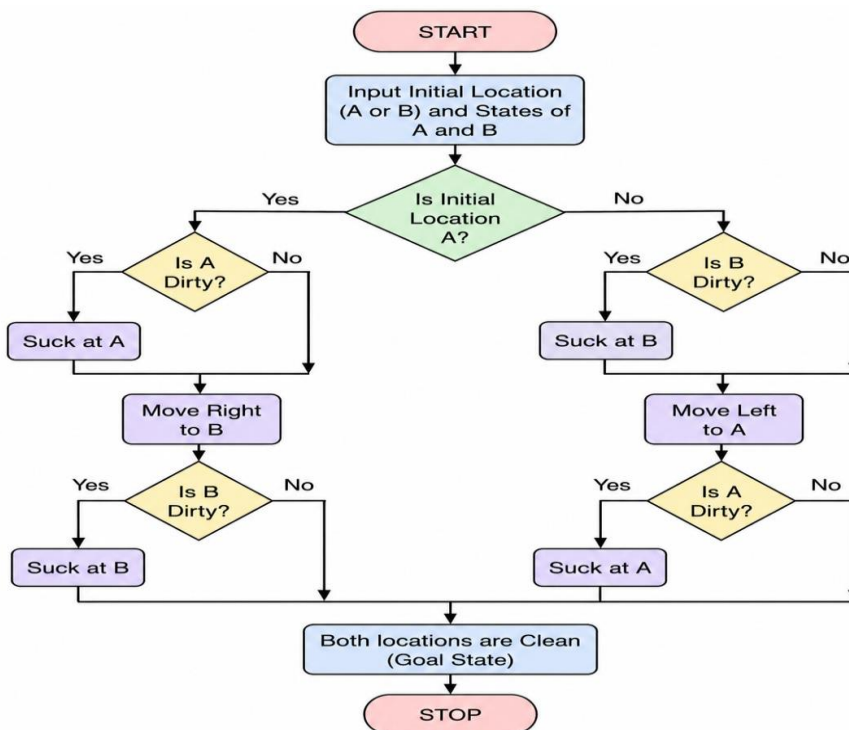
Objectives

The objective of the Vacuum Cleaner Problem is to design an intelligent agent that can sense whether a location is clean or dirty and perform the appropriate action to clean it. The agent uses actions such as Suck, Move Left, and Move Right to clean all dirty locations efficiently. The main goal is to ensure that the entire environment becomes clean using the minimum number of actions.

Algorithm :

1. Start.
2. Place the vacuum cleaner at the initial location (A or B).
3. Check whether the current location is Dirty or Clean.
4. If the location is Dirty, perform the Suck operation.
5. Move to the other location.
6. Check whether the new location is Dirty or Clean.
7. If the location is Dirty, perform the Suck operation.
8. Repeat until both locations are Clean.
9. Stop.

Flow chart :



Source Code :

```
location = input().upper()
A = input().capitalize()
B = input().capitalize()

if location == "A":
    if A == "Dirty":
        A = "Clean"
        print("Suck")
    print("Right")
    if B == "Dirty":
        B = "Clean"
        print("Suck")

elif location == "B":
    if B == "Dirty":
        B = "Clean"
        print("Suck")
    print("Left")
    if A == "Dirty":
        A = "Clean"
        print("Suck")

print(A)
print(B)
```

Result :

The Vacuum Cleaner Problem was successfully implemented. The vacuum cleaner agent cleaned all dirty locations and reached the goal state where both locations are clean.

Output :

main.py



Run

Output

Clear

```
1 location = "A"
2 A = "Dirty"
3 B = "Dirty"
4
5 if location == "A":
6     if A == "Dirty":
7         print("Suck at A")
8         A = "Clean"
9
10    print("Move Right")
11
12    if B == "Dirty":
13        print("Suck at B")
14        B = "Clean"
15
16 else:
17     if B == "Dirty":
18         print("Suck at B")
19         B = "Clean"
20
21    print("Move Left")
22
23    if A == "Dirty":
24        print("Suck at A")
25        A = "Clean"
26
27 print("Goal Achieved: Both locations are clean")
```

Suck at A
Move Right
Suck at B
Goal Achieved: Both locations are clean

=== Code Execution Successful ===

